INFORME DE PROYECTO

1. Información general

Nombre del proyecto: Monitoreo de Variables Ambientales (MVA)

Integrantes del equipo y roles

- Camilo José Ardila — Backend developer / QA
- Santiago Andrés Martínez — Backend developer
- Nicolás Meneses — Data scientist
- Luis Ángel Seoanes — Data scientist / Lead project

Docente / Evaluador: Claudia Stella Carmona Rodríguez

Periodo del proyecto: 13/02/2026 – 04/05/2026 (5 sprints quincenales)

Fecha del informe: 19/05/2026

2. Contexto y problema

Descripción del contexto

Medellín y su área metropolitana presentan un régimen climático bimodal con alta volatilidad diaria, marcado por un ciclo convectivo diurno que produce eventos de lluvia frecuentes y poco previsibles para el ciudadano. La movilidad cotidiana —motociclistas, ciclistas, peatones y usuarios de transporte público— se ve directamente afectada por estas condiciones, sin contar con una herramienta local que integre datos ambientales operativos con recomendaciones aterrizadas al Valle de Aburrá.

Problema identificado

Las soluciones públicas existentes (pronósticos genéricos, apps de navegación) no integran variables ambientales locales ni traducen la información meteorológica en recomendaciones accionables de movilidad para el usuario final. El usuario recibe datos crudos sin contexto interpretativo, lo cual no apoya decisiones preventivas en sus desplazamientos.

Necesidad del usuario/cliente

Disponer de una aplicación que combine pronóstico climático localizado, variables ambientales en tiempo real y enrutamiento, entregando recomendaciones instructivas (qué hacer ante una alerta) y no meramente sugestivas, para apoyar decisiones preventivas en el desplazamiento diario.

Importancia del problema

La movilidad en el Valle de Aburrá está fuertemente condicionada por la lluvia y por variables como visibilidad, humedad y temperatura. Una herramienta que reduzca la incertidumbre meteorológica del usuario impacta directamente en seguridad vial, productividad y bienestar, especialmente para poblaciones que dependen de vehículos de dos ruedas o transporte intermodal.

3. Objetivo del proyecto (OKR)

Objective: Desarrollar una solución que permita interpretar variables ambientales para apoyar decisiones de movilidad de los usuarios en el área metropolitana de Medellín.

Key Results

KR	Métrica	Línea base	Meta	Resultado
KR1	% de usuarios que interpretan correctamente las recomendaciones en pruebas de uso	0%	≥ 70%	75%
KR2	% de predicciones correctas de lluvia (F1-score) vs. predicción pública de referencia	F1 = 0.65 (modelo base)	+2% vs. referencia pública	F1 = 0.78
KR3	Número de variables ambientales integradas y operativas	0	≥ 6	8
KR4	% de usuarios que afirman que la app influyó en su decisión de movilidad	0%	≥ 40%	35% (parcial)

4. Alcance del proyecto

Qué incluye

- Aplicación web responsive (mobile-first) con autenticación JWT y vistas de navegación, planificación de viajes y alertas dinámicas.
- Backend en Java con JOOQ, contenerizado en Docker, con endpoints REST para ambiente, viajes y autenticación.
- Base de datos relacional con migraciones automatizadas, persistencia de viajes y condiciones ambientales.
- Modelos de Machine Learning para predicción de lluvia binaria (llueve / no llueve) entrenados con datos del SIATA y Open-Meteo.
- Integración con Google Maps API (enrutamiento) y Google Weather API (datos climáticos).
- Sistema de recomendaciones y alertas instructivas para movilidad, basadas en predicciones locales.
- Caracterización de usuario final y metodología de encuestas para validación UX.

Qué NO incluye

- Aplicación nativa para iOS o Android (el alcance se limita a una webapp responsive).
- Cobertura geográfica fuera del Valle de Aburrá.
- Predicción de variables ambientales distintas a lluvia (por ejemplo, calidad del aire en granularidad alta, sismos, inundaciones específicas).
- Despliegue en producción comercial; el entregable es un prototipo funcional con CI/CD parcial.
- Modelo de monetización, pasarela de pagos o gestión de planes.

Supuestos

- El SIATA y/o Open-Meteo continúan ofreciendo acceso a datos ambientales en la ventana del proyecto.
- Las API gratuitas de Google Cloud son suficientes para el volumen de pruebas del prototipo.
- El usuario final del MVP cuenta con conexión a internet móvil y un dispositivo con navegador moderno.
- La lluvia es el evento ambiental con mayor incidencia en decisiones de movilidad en Medellín.

5. Solución propuesta

Descripción general

Aplicación web de monitoreo ambiental y enrutamiento que integra datos del SIATA, Open-Meteo y Google Cloud para entregar al usuario predicciones de lluvia localizadas, recomendaciones instructivas de movilidad y un sistema de alertas dinámicas dentro de un flujo de planificación de viajes.

Arquitectura del sistema

Arquitectura por capas, totalmente contenerizada con Docker:

- Capa de presentación: Frontend web responsive (mobile-first) con vista principal ambiental, vista de rutas, vista de navegación con alertas, perfil de usuario y formularios UX.
- Capa de aplicación: Backend Java con endpoints REST (/api/ambiente, /api/viajes, /api/auth) y autenticación JWT.
- Capa de datos: Base de datos relacional con esquema versionado por migraciones (Flyway/Liquibase), modelos de ML serializados (.pkl) servidos vía endpoint dedicado.
- Capa de integración: Conectores a SIATA, Open-Meteo, Google Maps API y Google Weather API.

Tecnologías utilizadas

- Backend: Java + JOOQ, Docker.
- Frontend: HTML/CSS/JS responsive, mobile-first.
- Data Science: Python, pandas, scikit-learn, modelos serializados con pickle.
- Infraestructura: Docker Compose para orquestación local; CI/CD parcial.
- APIs externas: SIATA, Open-Meteo, Google Maps, Google Weather.
- Gestión: GitHub (repositorio único), Trello (tablero Kanban), Microsoft Teams.

Flujo del sistema

- El usuario se registra/autentica (JWT).
- Ingresa origen y destino; el frontend consume /api/viajes.
- El backend consulta Google Maps API para ruta, distancia y duración, y solicita predicción local al servicio ML.
- El modelo de lluvia evalúa condiciones ambientales actuales (SIATA + Open-Meteo) y devuelve probabilidad de lluvia.
- El backend compone la respuesta: ruta + condiciones + recomendación instructiva + alertas.
- El frontend presenta la vista de navegación con mapa, alertas activas y banderas de movilidad.
- El viaje y sus condiciones quedan persistidos para análisis histórico.

6. Desarrollo por sprints

Sprint	Objetivo	Entregables	Resultado	KR impactados
S1 13/02–27/02	Descubrimiento, definición y base técnica.	Backend inicial, BD inicial, pipeline de limpieza de datos, primera variable ambiental integrada.	Logrado	KR1, KR3
S2 28/02–13/03	MVP funcional con variables ambientales.	Backend con Docker y servidor; modelos base de predicción	Parcial	KR1, KR2, KR3

Sprint	Objetivo	Entregables	Resultado	KR impactados
		(F1=0.65); 8 variables vía Open-Meteo.		
S3 26/03–06/04	Servicios base, integración SIATA, enrutamiento.	Backend funcional, modelos mejorados (F1=0.75), contenedores BD+Backend+ML+Frontend, enrutamiento.	Logrado	KR1, KR2, KR3, KR4
S4 06/04–20/04	Integrar componentes hacia una UI interpretable y funcional.	UI funcional con planificación de viajes, autenticación integrada, modelo refinado (F1=0.78), feedback inicial.	Logrado	KR1, KR4
S5 20/04–04/05	Mejorar UX, alertas dinámicas y validación con usuarios.	Alertas instructivas, mejoras de UI/UX, perfil de usuario, validación por encuesta.	Logrado	KR1, KR4

Logros principales

- Sistema contenerizado end-to-end (BD + Backend + ML + Frontend) reproducible en cualquier entorno con Docker.
- Modelo de predicción de lluvia mejoró de F1=0.65 a F1=0.78 con la incorporación de datos del SIATA y ajustes de variables.
- Integración exitosa con Google Cloud (Maps + Weather) como contingencia ante latencia en datos del SIATA.
- Migración del backlog desde sugerencias pasivas hacia alertas instructivas accionables para el usuario.
- Caracterización del usuario final y aplicación de metodología de encuesta para validar comprensión y utilidad.

Dificultades

- Lentitud en la entrega de credenciales y datos del SIATA durante los primeros sprints; bloqueante recurrente.
- Separación inicial entre los equipos de Data Science y Backend, generando redundancia y sobreescritura de código.
- Sesgo de diseño en la UI inicial: el equipo asumió alta interpretabilidad sin validar con usuarios externos.
- Opacamiento del modelo propio por la facilidad de uso de Google Weather API, desviando el foco local.
- Riesgo de costos por sobreconsumo de APIs gratuitas de Google si no se controla el volumen de peticiones.

Cómo se resolvieron

- Contingencia con Open-Meteo y Google Weather mientras llegaban los datos del SIATA, lo que evitó detener el desarrollo.
- Weekly fijo los viernes 3 PM, unificación del trabajo en un solo repositorio de GitHub y en Trello como tablero único.
- Validación temprana de UI con usuarios externos (HU033 — formulario UX) y rediseño basado en feedback piloto.

- En el Sprint 5 se reactivó el rol protagónico del modelo propio frente a la dependencia de Google, alineando la solución al énfasis local.
- Limpieza general del repositorio, merges estructurados y monitoreo de llamadas a Google API para mantenerse en cuota gratuita.

7. Backlog del proyecto

Historias de usuario principales (consolidado del proyecto)

ID	Historia de usuario	Prioridad	Estado
001	Integración de datos ambientales de prueba	MEDIA	Hecho
002	Normalización y organización de variables ambientales	ALTA	Hecho
003	Integración de ≥3 variables ambientales en dataset de entrenamiento	ALTA	Hecho
004	Desarrollo base del backend	ALTA	Hecho
005	Construcción de base de datos para lógica de negocio	ALTA	Hecho
006	Lógica de viajes vía Google Maps API	ALTA	Hecho
007	Fix JOOQ — comunicación estable con BD	ALTA	Hecho
008	Fix API OpenWeather para testing	MEDIA	Hecho
009	Docker y servidor — stack completo	BAJA	Hecho
010	Definir benchmark público de predicción	BAJA	Hecho
011	Modelo mejorado de predicción de lluvia	ALTA	Hecho
012	Frontend: pantalla principal ambiental	ALTA	Hecho
013	Sistema de enrutamiento en Medellín	ALTA	Hecho
014	Endpoint REST /api/ambiente	ALTA	Hecho
015	Historial de viajes en BD	ALTA	Hecho
016	Autenticación JWT (registro + login)	ALTA	Hecho
017	Esquema de BD completo con migraciones	ALTA	Hecho
018	Integración backend ↔ modelo de lluvia	ALTA	Hecho
019	QA: pruebas de integración del backend	ALTA	Hecho
021	Vista principal del usuario	ALTA	Hecho (requiere revisión menor)
022	Vista de rutas y planificación de viajes	ALTA	Hecho (requiere revisión menor)
023	Integración de funciones con autenticación	ALTA	Hecho
024	Pruebas unitarias a nivel de código	ALTA	En progreso
025	Despliegue con CI/CD	MEDIA	En progreso (detalles menores)

ID	Historia de usuario	Prioridad	Estado
026	Pruebas piloto con usuarios	ALTA	En progreso
027	Formulario de preguntas para validar UX	MEDIA	Hecho
028	Integración del modelo con API de Google	ALTA	Hecho
029	Ajuste de variables y reglas del modelo para despliegue	ALTA	Hecho
030	Mejoras generales al frontend	ALTA	Hecho (detalles menores)
031	Recomendaciones activas y alertas de movilidad	MEDIA	Hecho (detalles menores)
032	Recomendaciones con múltiples destinos en un solo viaje	ALTA	Refactorización posible
033	Validar comprensión del formulario de UX	ALTA	Hecho
034	Análisis y sugerencias de mejoras en UI	ALTA	Hecho (mejoras menores)
035	Integración del modelo ajustado en tiempo real	ALTA	Hecho
036	Mapa de calor sobre la navegación	MEDIA	Pendiente
037	Centrado y giro automático en la vista de navegación	MEDIA	Pendiente

Priorización

Criterio principal: impacto directo en la comprensión del usuario (KR1) y en la dependencia técnica de componentes críticos (por ejemplo, la lógica de viajes depende del enrutamiento, que a su vez depende del modelo). En sprints finales se priorizaron tareas de frontend y funcionalidades que maximizan el valor percibido por el usuario sobre los casos de uso reales.

8. Resultados obtenidos

Qué se construyó

Una webapp funcional y contenerizada de monitoreo ambiental y enrutamiento para el área metropolitana de Medellín. Integra autenticación JWT, una base de datos relacional con migraciones, un backend Java con endpoints REST, modelos de predicción de lluvia entrenados con datos del SIATA y Open-Meteo, conectores con Google Maps y Google Weather API, una interfaz responsive con vistas de navegación, planificación de viajes, alertas instructivas y formularios de validación UX.

Evidencias

- Repositorio único en GitHub con código de backend, frontend, notebooks de Data Science y configuración Docker.
- Archivos .pkl de modelos entrenados versionados en el repositorio.
- Tablero Trello con historias completadas y trazabilidad por sprint.
- Stack contenerizado ejecutable con un solo comando (Docker Compose).
- Encuestas aplicadas a usuarios piloto y resultados de validación UX.

Comparación con OKR

KR	Meta	Resultado	Cumplimiento
KR1: Comprensión del usuario	≥ 70%	75%	Cumplido
KR2: Precisión del modelo (F1)	+2% vs. referencia	F1 = 0.78	Cumplido
KR3: Variables ambientales integradas	≥ 6	8	Cumplido
KR4: Influencia en decisiones de movilidad	≥ 40%	35%	Parcial — depende del volumen de encuestas

9. Validación de la solución

Pruebas realizadas

- Pruebas de integración backend (HU019): validación de todos los endpoints REST del proyecto.
- Pruebas unitarias parciales sobre componentes críticos del backend y del modelo ML.
- Pruebas piloto con usuarios externos al equipo, con metodología de encuesta y caracterización previa del perfil de usuario final.
- Validación del formulario UX (HU033) con usuarios reales para evaluar comprensión de la interfaz y de las alertas.
- Validación cuantitativa del modelo de predicción de lluvia con métricas F1, accuracy y comparación contra Open-Meteo como referencia pública.

Resultados

- Modelo de predicción: F1 = 0.78 (mejora sostenida desde F1 = 0.65 en S2).
- Interpretación correcta de recomendaciones por usuarios piloto: 75% (KR1).
- Usuarios que reportan haber sido influidos por la app en sus decisiones de movilidad: 35% (KR4).
- Endpoints del backend validados como estables en pruebas de integración.
- Feedback explícito de usuarios sobre paleta de colores y visibilidad de la UI bajo distintas condiciones de iluminación, incorporado en mejoras de S5.

Limitaciones

- Población encuestada relativamente pequeña; ampliar la muestra es necesario para consolidar el KR4.
- Modelo entrenado con un horizonte temporal limitado de datos del SIATA; su generalización en estaciones extremas no fue evaluada formalmente.
- Cobertura geográfica restringida al área metropolitana de Medellín.
- CI/CD parcial; el despliegue productivo continuo no está cerrado.
- Dependencia de APIs externas (Google, SIATA, Open-Meteo) que pueden cambiar términos, costos o disponibilidad.

10. Uso de inteligencia artificial

Uso planificado

- Claude / Claude Code: generación de código para prototipos de frontend y notebooks de modelos.

- ChatGPT: documentación, estructuración de backlog, planificación de sprints e interpretación de estadística descriptiva sobre los datasets.
- Grok: apoyo a llamadas a API (Open-Weather, Google Maps) y scripts de web scraping y organización de datos.

Uso real

La IA se empleó de manera continua a lo largo del proyecto, principalmente para acelerar tareas tediosas: generación de boilerplate de frontend, scaffolding de notebooks, redacción inicial de documentación, traducción de diagramas a código, generación de scripts utilitarios y propuestas de estructura para historias de usuario. En ningún sprint se ingresaron datos sensibles ni se delegó a la IA la toma de decisiones técnicas críticas.

Reflexión crítica

La IA generativa actúa como un multiplicador de productividad cuando el usuario posee el conocimiento técnico para evaluar críticamente lo que produce. En tareas de UI/UX el equipo identificó que las propuestas de la IA tienden a ignorar restricciones del mundo real (iluminación, visibilidad en exteriores, tamaño de pantalla); en tareas de arquitectura, suele sugerir servicios privados "estándar" de alta complejidad por sesgo de entrenamiento. La trazabilidad del trabajo asistido por IA se degrada si no se documenta explícitamente qué partes fueron generadas o reformuladas con su apoyo, lo cual se mitigó mediante revisiones por pares y commits descriptivos.

Consideraciones éticas

- No se ingresaron datos sensibles ni confidenciales en ninguna herramienta de IA.
- Se verificó y validó críticamente la información generada antes de incorporarla al proyecto.
- Se documentó qué partes del trabajo fueron apoyadas por IA en cada informe de sprint.
- La IA no reemplazó el análisis crítico del equipo en decisiones de arquitectura, modelado ni validación de resultados.
- Se respetaron principios de privacidad, propiedad intelectual y transparencia.
- Riesgos identificados y mitigados: falta de explicabilidad de los modelos, alucinaciones, dependencia excesiva, pérdida de autonomía técnica del desarrollador y posibles afectaciones a derechos de autor.

11. Evolución del sistema

Estado actual del prototipo

Aplicación web funcional, contenerizada, con autenticación JWT, BD relacional, backend Java con endpoints REST, modelos ML de predicción de lluvia (F1=0.78), frontend responsive con vistas de navegación, planificación de viajes, alertas instructivas y módulos de feedback. Despliegue con CI/CD parcial; pruebas unitarias y piloto en curso.

Brechas identificadas

- CI/CD aún no cerrado completamente (HU025 con detalles menores pendientes).
- Cobertura de pruebas unitarias incompleta (HU024 en progreso).
- Pruebas piloto con una muestra reducida; se requiere ampliar para consolidar KR4.
- Refactor pendiente para soportar múltiples destinos en un solo viaje (HU032).
- Funcionalidades visuales como mapa de calor (HU036) y centrado/giro automático (HU037) aún no implementadas.

- Lógica de recomendaciones del modelo propio todavía parcialmente opacada por Google Weather API.

Fases futuras

- Fase 1 — Cierre del MVP: completar HU024, HU025, HU026 y refactor de HU032.
- Fase 2 — Expansión funcional: implementar mapa de calor (HU036) y mejoras de navegación (HU037), incorporar alertas para más fenómenos ambientales.
- Fase 3 — Ampliación de validación: encuestas a poblaciones más grandes y diversificadas para consolidar KR4 (≥ 40%).
- Fase 4 — Robustez del modelo: reentrenamiento periódico con nuevos datos del SIATA, calibración estacional, evaluación con un horizonte temporal mayor.
- Fase 5 — Producción: monitoreo de costos de APIs externas, despliegue en infraestructura productiva, internacionalización de la base de datos para nuevas ciudades.

Requerimientos para evolucionar

- Acceso sostenido y formalizado a datos del SIATA.
- Presupuesto o convenio para uso de Google Cloud Platform fuera de la cuota gratuita.
- Equipo dedicado a QA y a diseño de UI/UX para superar el sesgo de auto-validación.
- Infraestructura productiva con monitoreo, logs y observabilidad.

Riesgos

- Costos asociados al uso de APIs de Google si se sobrepasa la cuota gratuita.
- Cambios o suspensión del acceso a datos del SIATA.
- Dependencia técnica de proveedores externos (Google, Open-Meteo).
- Pérdida de interpretabilidad si el modelo propio sigue siendo desplazado por servicios externos genéricos.
- Sobreescritura de código y pérdida de trazabilidad en equipos distribuidos sin disciplina de comunicación.

12. Base documental del Proyecto

Los siguientes documentos se entregan como anexos independientes:

Informe técnico

- Arquitectura general del sistema (capas, módulos, contenedores).
- Tecnologías utilizadas en cada capa.
- Flujo de datos entre frontend, backend, modelo y APIs externas.
- Supuestos técnicos (disponibilidad de APIs, formato de datos, cuotas de servicios externos).

Manual de Usuario

- Descripción del sistema y casos de uso principales.
- Cómo usar la aplicación: registro, login, planificación de viaje, lectura de alertas.
- Flujos principales con capturas paso a paso.
- Ejemplos de escenarios cotidianos (motociclista, peatón, usuario de transporte público).

Manual de pruebas

- Qué se probó: endpoints, modelo ML, UI, flujo completo.

- Cómo se probó: pruebas de integración, pruebas unitarias, pruebas piloto con usuarios y encuesta UX.
- Resultados obtenidos: F1=0.78 en modelo, 75% comprensión, 35% influencia reportada, estabilidad de endpoints.

13. Trabajo en equipo

Distribución de tareas

Integrante	Área principal	Responsabilidades clave
Camilo José Ardila	Backend / QA	Lógica de viajes (Google Maps), endpoints REST, persistencia de viajes, QA del backend, integración OpenWeather, frontend (apoyo).
Santiago Andrés Martínez	Backend	Configuración Docker y servidor, JOOQ, autenticación JWT, esquema de BD con migraciones, integración backend-ML, CI/CD.
Nicolás Meneses	Data Science	Modelado de predicción de lluvia, ingeniería de features, validación de calidad de datos, formularios UX, frontend (apoyo), benchmark público.
Luis Ángel Seoanes	Data Science (Lead)	Lead del proyecto, enrutamiento, modelo ML, integración del modelo en tiempo real, ajuste de variables del modelo para despliegue.

Aportes individuales destacados

- Camilo Ardila: lógica de viajes completa con Google Maps API, QA sostenido a lo largo del proyecto, prototipos de frontend.
- Santiago Martínez: solución de los bloqueos de JOOQ que destrabaron la comunicación con la BD; autenticación JWT y esquema de migraciones.
- Nicolás Meneses: pipeline de calidad de datos, modelado base, definición del benchmark público, formularios de validación UX.
- Luis Seoanes: liderazgo del proyecto, sistema de enrutamiento integrado con condiciones ambientales, ajuste fino del modelo para despliegue.

Dificultades

- Separación inicial entre Data Science y Backend que generó redundancia y sobreescritura.
- Disponibilidad variable de los integrantes por carga académica y asuntos personales (especialmente en S4 y S5).
- Coordinación de ramas de Git: acumulación de avances dispersos y código obsoleto antes de la limpieza del S5.

Soluciones

- Weekly fijo los viernes 3 PM como punto de sincronización.
- Unificación del trabajo en un solo repositorio de GitHub y un solo tablero Trello.
- Comunicación explícita previa a cada tarea: cada integrante anuncia qué tocará y qué dependencias introduce.
- Limpieza estructurada del repositorio en S5 (merges, eliminación de ramas obsoletas, organización por módulos).

14. Lecciones aprendidas

Técnicas

- Contenerizar el stack temprano paga dividendos: redujo la fricción de onboarding y de pruebas integradas.
- No subordinar el modelo propio a servicios externos genéricos: la facilidad de Google Weather opacaba el valor diferencial local.
- Los modelos de ML deben validarse con métricas comparables (F1) y con un benchmark externo definido desde el inicio.
- CI/CD debe asegurarse temprano, no al final del proyecto.
- Las migraciones de BD versionadas evitan inconsistencias entre entornos del equipo.

De gestión

- Las dependencias externas críticas (como datos del SIATA) deben tener plan B desde el primer sprint.
- La capacidad estimada debe calibrarse durante los dos primeros sprints; planificar como si el ritmo final fuera el inicial subestima el trabajo.
- Documentar acuerdos en cada retrospectiva y aplicarlos: la mayoría de los compromisos del S1 al S5 se cumplieron porque quedaron explícitos en los formatos de seguimiento.
- El backlog debe limpiarse y repriorizarse cada sprint; no es un archivo acumulativo.

De equipo

- La comunicación explícita previa a cada tarea evita sobreescritura y redundancia.
- Las áreas técnicas (DS vs Backend) deben acordar interfaces tempranas y no trabajar en silos.
- Validar la UI con personas externas al equipo es indispensable; los autores no son buenos jueces de su propia interfaz.
- El repositorio debe reflejar el estado actual del proyecto, no la historia entera; Git ya conserva el pasado.

Sobre IA

- La IA generativa acelera tareas tediosas, pero requiere criterio humano para decisiones de arquitectura y validación.
- Las propuestas de UI de la IA suelen ignorar restricciones del mundo real (iluminación, visibilidad, tamaño de pantalla).
- La IA tiende a sobrediseñar (servicios privados, complejidad innecesaria); el equipo debe filtrar según el contexto real del proyecto.
- Documentar qué partes del trabajo fueron apoyadas por IA es indispensable para la trazabilidad.
- La IA no debe reemplazar el juicio crítico ni el análisis del equipo en decisiones técnicas relevantes.

15. Conclusiones

Valor del proyecto

El proyecto entrega una aplicación web funcional y contenerizada que demuestra la viabilidad técnica de integrar datos ambientales locales con enrutamiento y recomendaciones de movilidad para el área metropolitana de Medellín. La solución cubre todas las capas (datos, lógica, presentación) y alcanza los KR1, KR2 y KR3 dentro del horizonte del semestre.

Impacto logrado

Cuatro de los cuatro Key Results muestran avance significativo, con tres plenamente cumplidos. El modelo de predicción de lluvia mejoró 20% en F1 desde la línea base, el 75% de los usuarios piloto interpretan correctamente las recomendaciones, y el 35% afirma que la app influyó en sus decisiones de movilidad. Más allá de las métricas, el equipo consolidó una forma de trabajo basada en comunicación explícita, contenerización temprana, validación con usuarios externos y uso responsable de IA.

Proyección futura

El siguiente horizonte natural del proyecto es cerrar el MVP (CI/CD completo, cobertura de pruebas, refactor de viajes multidestino), ampliar la validación con un muestreo más amplio para consolidar KR4, fortalecer el rol del modelo propio frente a servicios externos genéricos y preparar un despliegue productivo con monitoreo de costos. A más largo plazo, el sistema puede extenderse a nuevas ciudades y a más fenómenos ambientales (calidad del aire, eventos climáticos extremos), siempre manteniendo el énfasis local que constituye su ventaja diferencial.

16. Anexos

Diagramas

- Diagrama de arquitectura del sistema (capas y contenedores).
- Diagrama de flujo de datos entre frontend, backend, modelo y APIs externas.
- Diagrama entidad-relación de la base de datos.

Código

- Repositorio único de GitHub con backend, frontend, notebooks de Data Science y configuración Docker.
- Archivos .pkl de modelos entrenados versionados.
- Docker Compose para orquestación local.

Capturas

- Capturas de la vista principal ambiental, vista de rutas, navegación con alertas y formulario UX.
- Capturas del tablero Trello al cierre de cada sprint.

Datos

- Dataset consolidado de variables ambientales (SIATA + Open-Meteo).
- Resultados de encuestas a usuarios piloto.
- Informes individuales de seguimiento quincenal de los 5 sprints.